Convolutional Neural Networks

in Keras

(CNN)

Kernel in practise: https://setosa.io/ev/image-kernels/

Convolution animations: https://github.com/vdumoulinconv_arithmetic

lesson 03

Convolution

function $f(x)$ (input)

filter function $g(x)$

$$C(x) = \int_a^b f(x-x')\, g(x')\, dx'$$

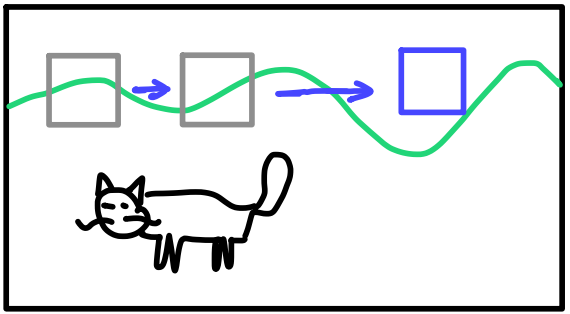
output feature map

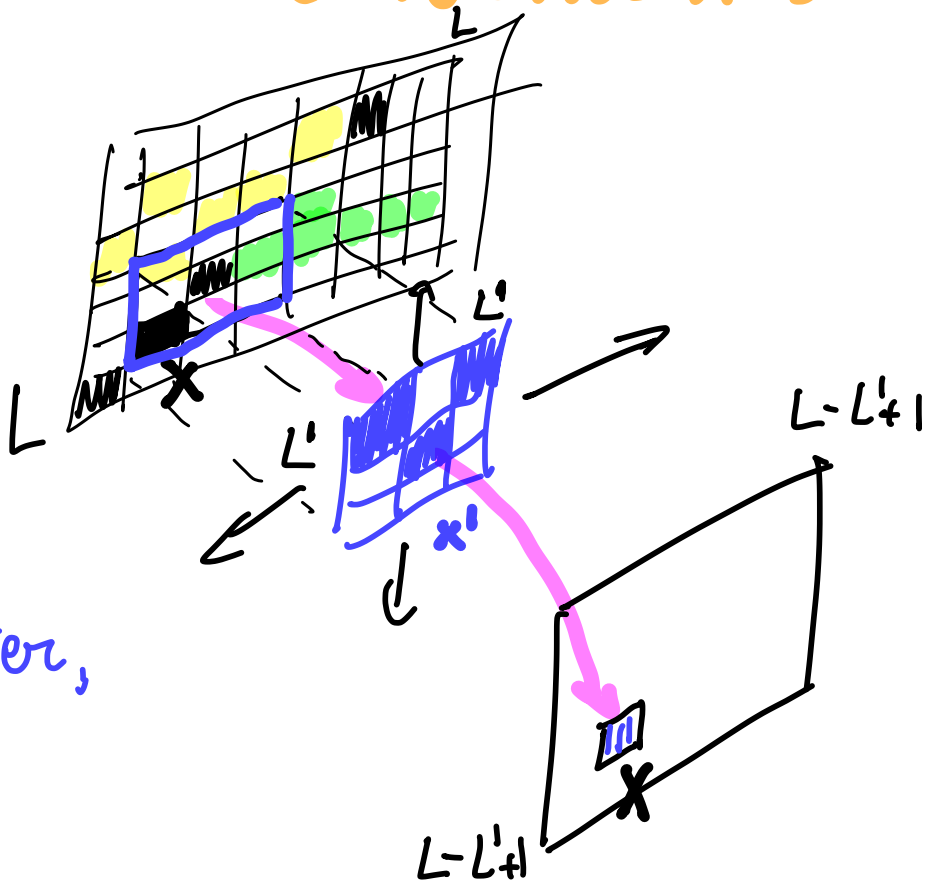
discrete 2D convolution $C(x) = C(i,j) = \sum_m \sum_n I(i-m, j-n)\, K(m,n)$

input image filter, kernel

Convolutional Neural Networks

in Keras

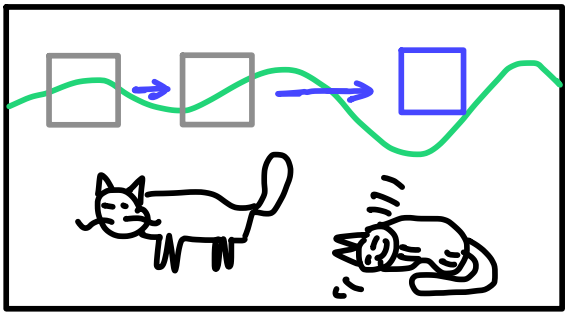




mask, filter, kernel ...

Convolutional Neural Networks

in Keras



- automatic definition of filters by CNN

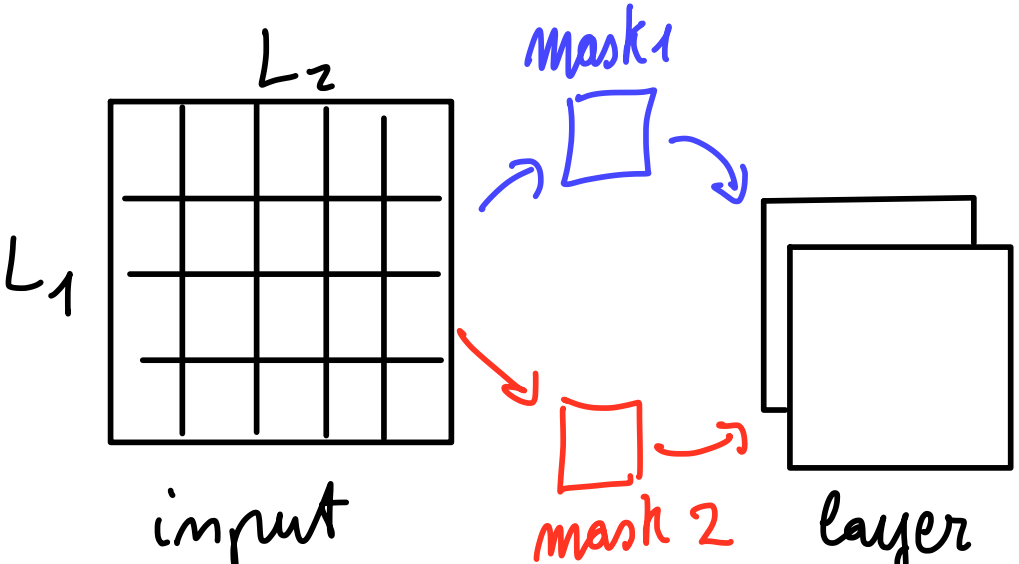
In CNNs filters are not specified manually.
Filters are randomly initialized and learnt during the training

$$\frac{\partial C}{\partial w_k}$$

mask 1

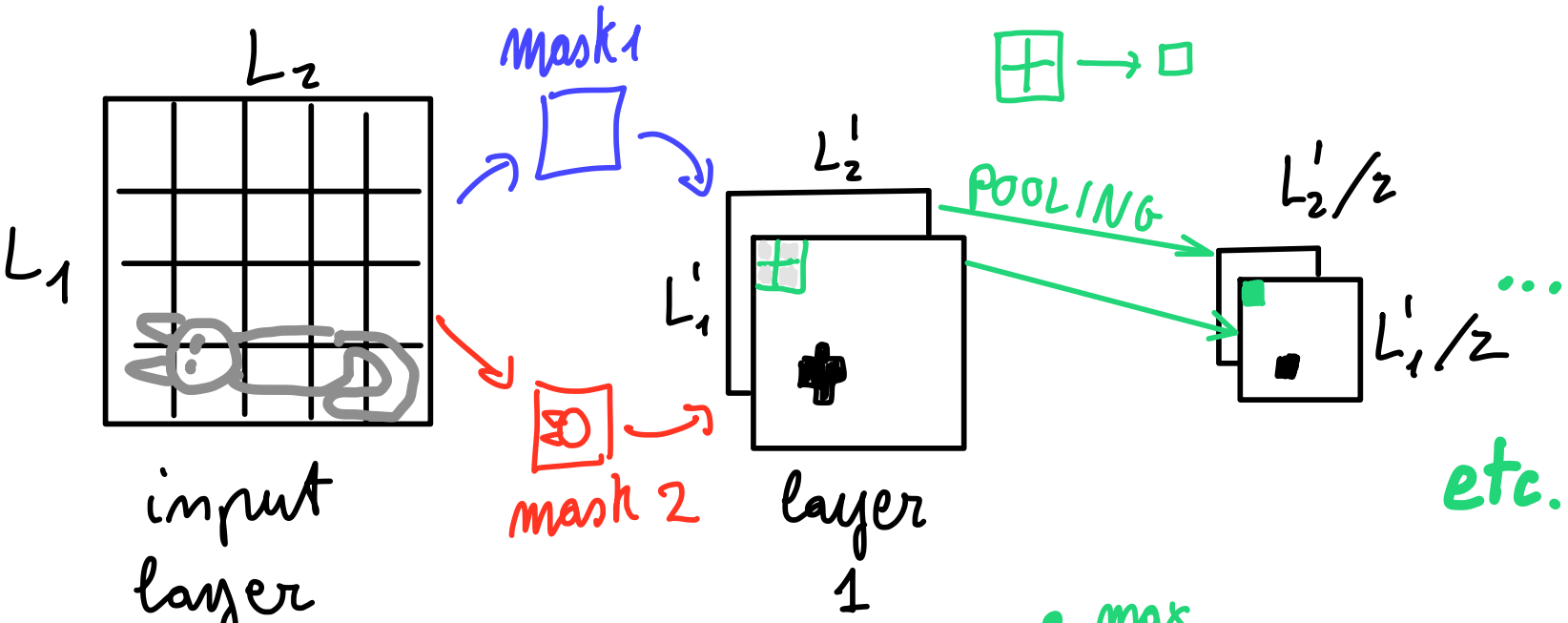
mask 2

multiple masks - multiple channels



$L_1 \times L_2$ (× channels)
R G B

- input image $H \times W \times C_{in}$
- filter spans all input channel ($K_h \times K_w \times C_{in}$)
- filter → feature map
- depth of output tensor = # filters



$L_1 \times L_2$ (× channels)
R G B

- max
- average
- ...

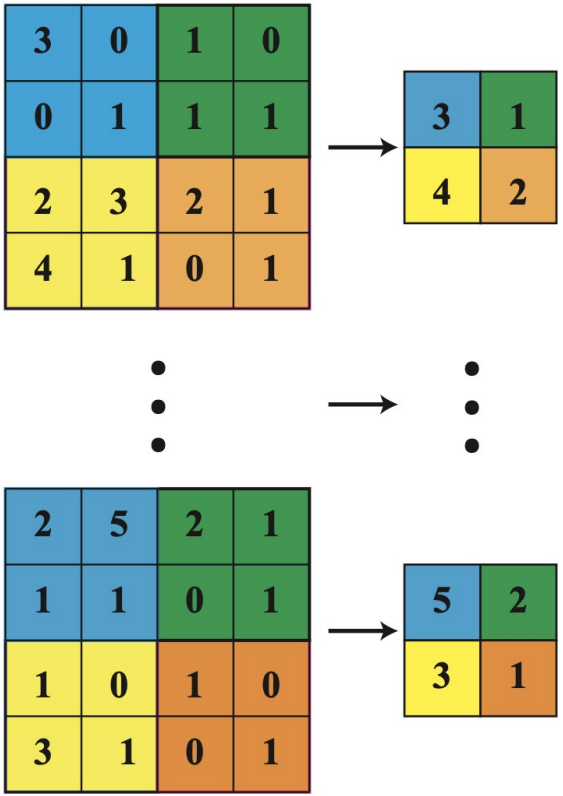


FIG. 44 **Illustration of Max Pooling**. Illustration of max-pooling over a 2×2 region. Notice that pooling is done at each depth (vertical axis) separately. The number of outputs is halved along each dimension due to this coarse-graining.

- pooling applied independently to each feature (downsampling)

- COARSE-GRAINING: Keeping dominant local activation while reducing resolution.

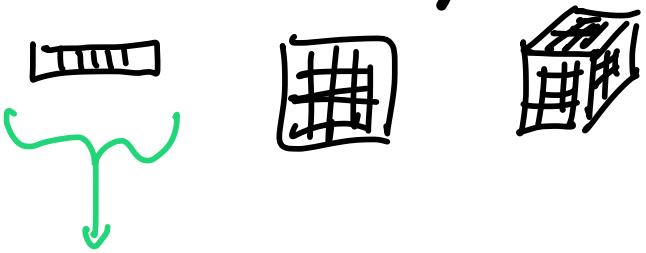
- convolution → local features
  pooling → compression & robustness

Why CNN?

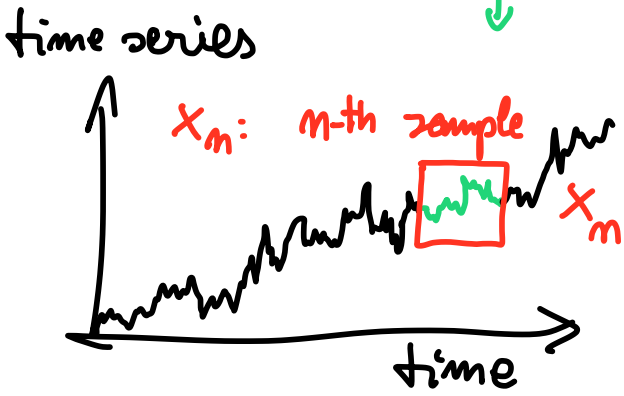
- learning fewer parameters than normal NN
  - more efficient
  - reduce overfitting?
- logic choice if we look for local structures in each data sample
  - spatial
  - temporal

(translational invariance)

CNN : 1D, 2D, 3D ...



↳ dimension of the convolution matches the dimensional organization of data.

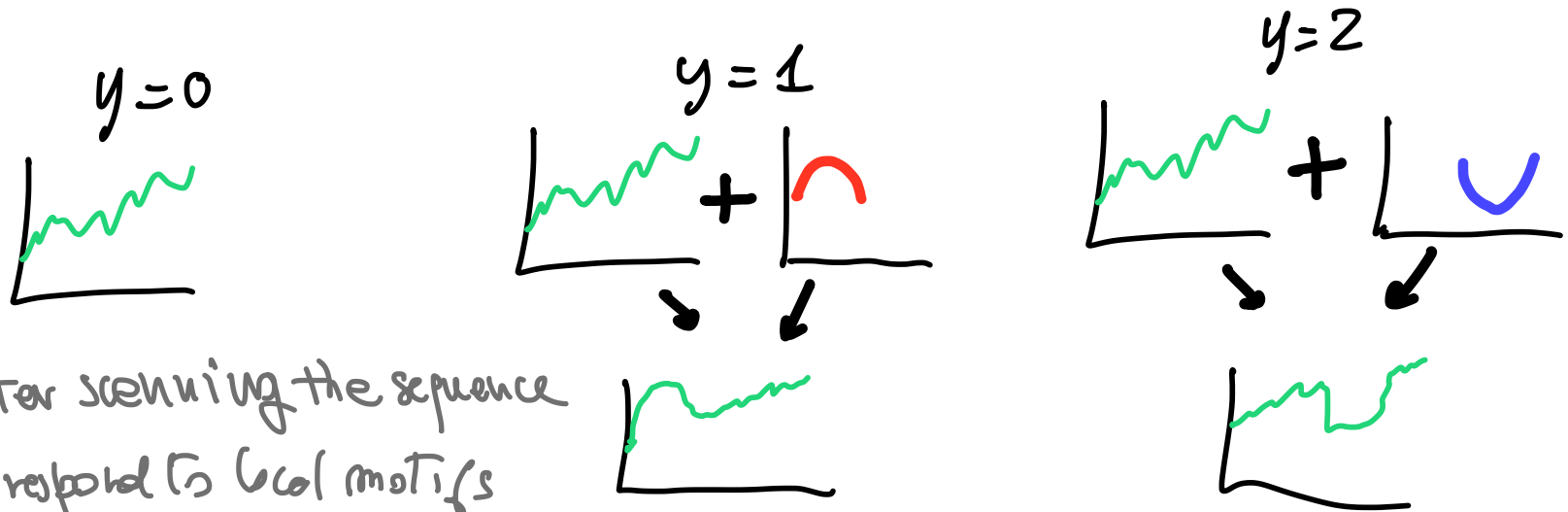


$$x_n = [x_n(0), \dots, x_n(L-1)]$$

exercise 3

- stochastic process : Markov chain $\quad X(t) \xrightarrow{P} X(t+1)$

- samples in 3 categories



filter scanning the sequence and responds to local motifs

Categories:

$M = 3$ categories $\qquad y = (0, 1, 2)$

$0 \to (1,0,0)$, $1 \to (0,1,0)$, $2 \to (0,0,1)$

cost function: softmax (eq (81) review)

(probability of each category) p

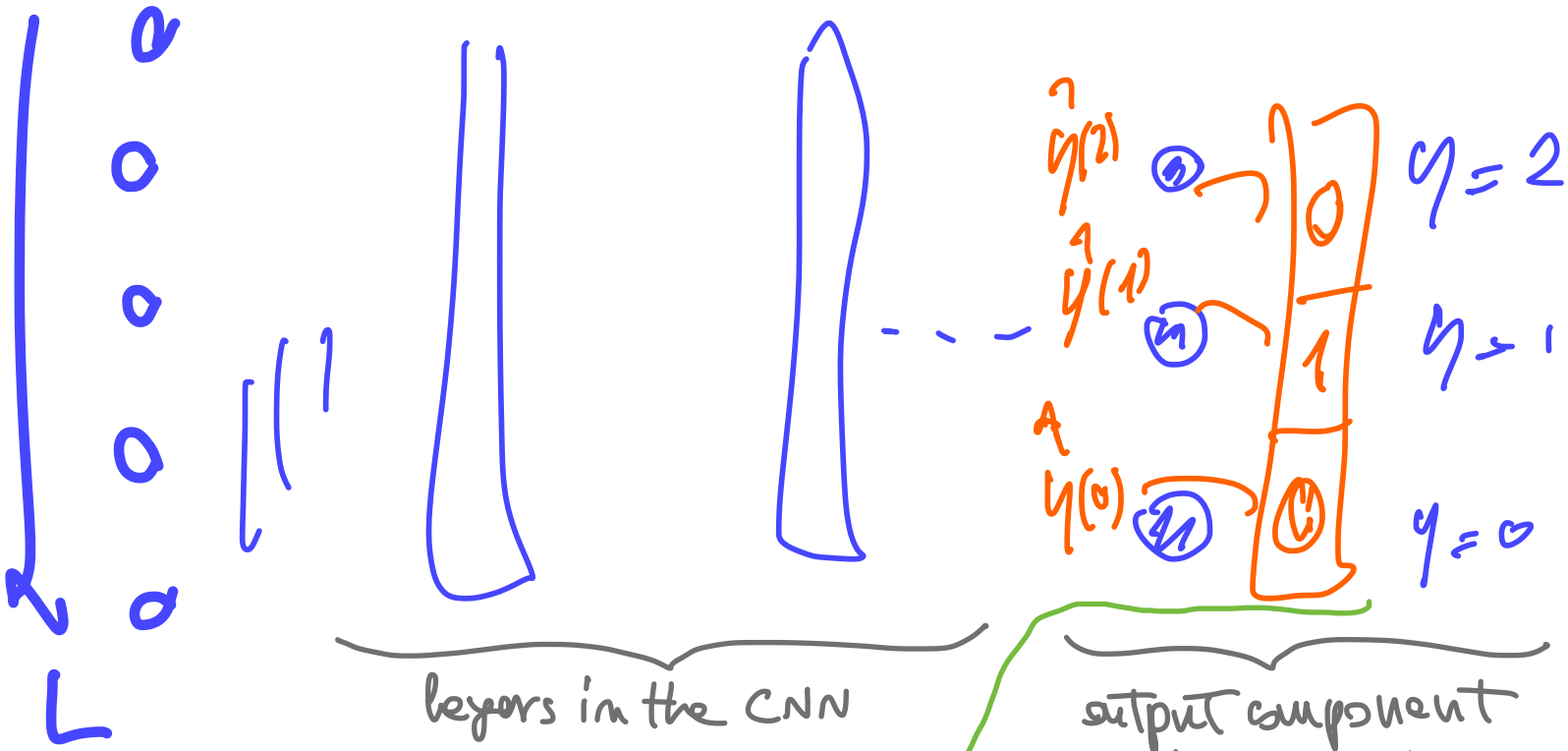
categorical cross entropy

$$C_\theta = -\sum_{n=1}^{N} \sum_{m=0}^{M-1} \left[y_{nm} \log p(y_{nm} | x_n) + (1 - y_{nm}) \log(1 - p(y_{nm} | x_n)) \right]$$

true label

predictor - output of CNN

from input to class scores



Categories:

$n: \quad y_n = (0, 1, 0)$

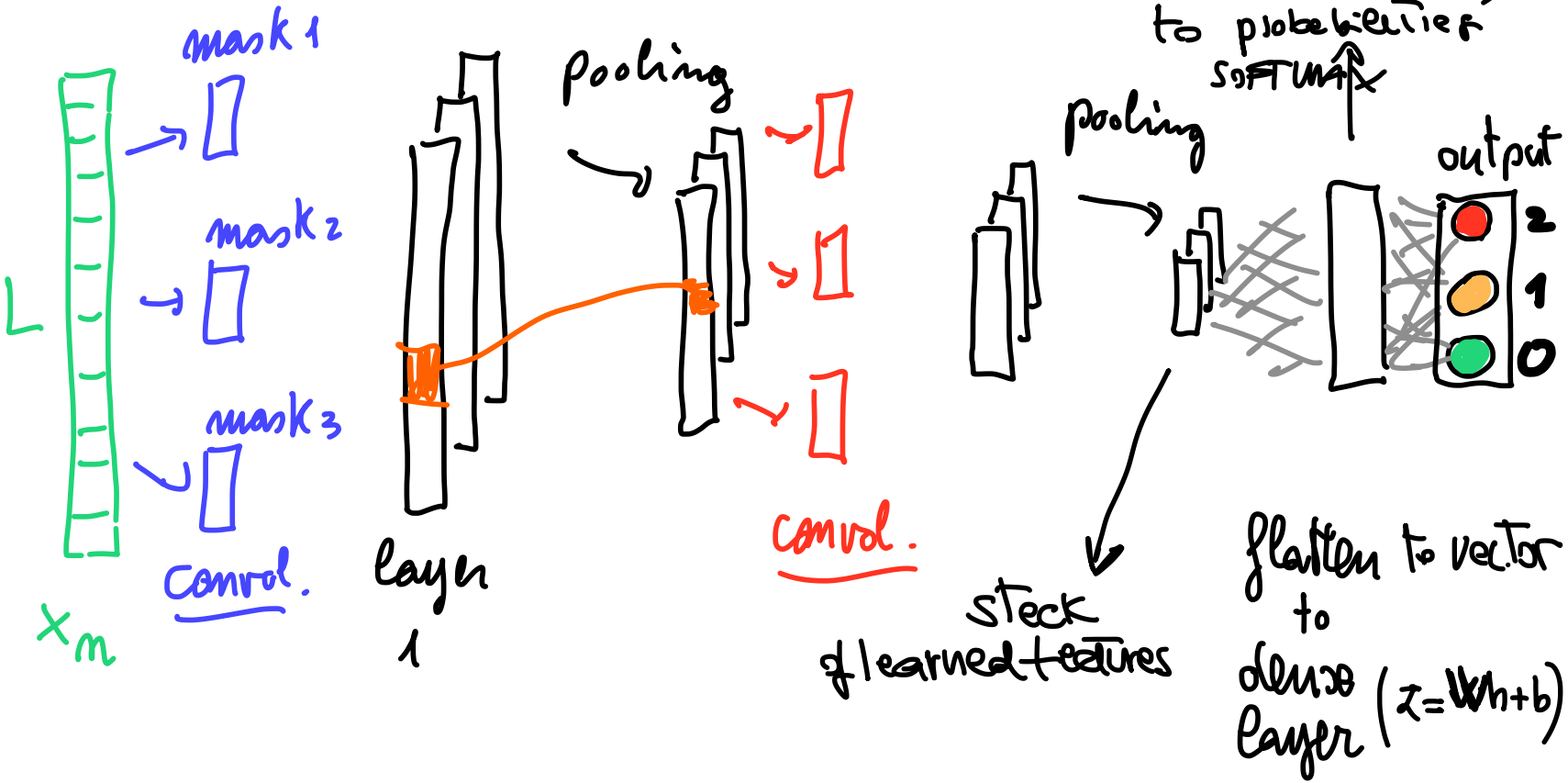
$P(y_n | x_n) = (0.2, \underline{0.5}, 0.3)$

$P(y_{nm} | x_n)$

θ = parameters (w, b)

$$C_\theta = -\sum_{n=1}^{N} \sum_{m=0}^{M-1} \left[y_{nm} \log P(y_{nm} | x_n) + (1 - y_{nm}) \log(1 - P(y_{nm} | x_n)) \right]$$

CNN recap


mask 1
mask 2
mask 3
L
x_n
convol.
layer 1
pooling
convol.
pooling
stack of learned features
flatten to vector to dense layer ($z = Wh + b$)
from scores $z = (z_0, z_1, z_2)$ to probabilities
SOFTMAX
output
2
1
0

CONVOLUTIONAL NEURAL NETWORKS (see Mehta, Sec. X)

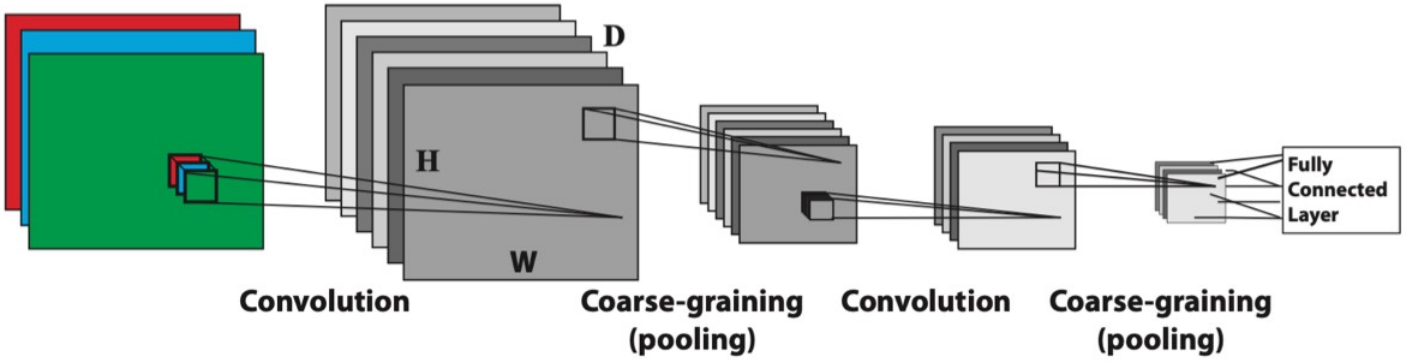


FIG. 42 **Architecture of a Convolutional Neural Network (CNN).** The neurons in a CNN are arranged in three dimensions: height (H), width (W), and depth (D). For the input layer, the depth corresponds to the number of channels (in this case 3 for RGB images). Neurons in the convolutional layers calculate the convolution of the image with a local spatial filter (e.g. 3×3 pixel grid, times 3 channels for first layer) at a given location in the spatial (W, H)-plane. The depth D of the convolutional layer corresponds to the number of filters used in the convolutional layer. Neurons at the same depth correspond to the same filter. Neurons in the convolutional layer mix inputs at different depths but preserve the spatial location. Pooling layers perform a spatial coarse graining (pooling step) at each depth to give a smaller height and width while preserving the depth. The convolutional and pooling layers are followed by a fully connected layer and classifier (not shown).

CNN

& previous exercise

A	C	G	A	T ....

L_s

$N_c = 4$

1000	0100	0010	1000	...

$L = L_s \cdot N_c$

$\Rightarrow$ standard DNN

+ data augmentation

CNN

& previous exercise

A	C	G	A	T ....

L_s

$N_c = 4$

1000	0100	0010	1000	...

$L = L_s \cdot N_c$

$\Rightarrow$ standard DNN

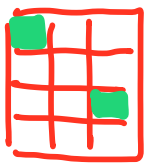
+ data augmentation

N_c

1	0	0	1	
0	1	0	0	....
0	0	1	0	
0	0	0	0	

L_s

$\Rightarrow$ 2D CNN, no augment.

mask 

if ...A.G.... key was required

Regularization:

cost function (loss) $C_\theta \longrightarrow \underbrace{C_\theta + \lambda R_\theta}_{\text{to be minimized}}$ control overfitting

- <u>Ridge</u> : $R_\theta = \sum_i |\theta_i|^2$ (θ_i = one of the weights in W, b)

- <u>LASSO</u> : $R_\theta = \sum_i |\theta_i|$

λ: multiplier, tunes strength of regulariz.

Regularization:

cost function $C_\theta \longrightarrow \underbrace{C_\theta + \lambda R_\theta}_{\text{to be minimized}}$

- <u>Ridge</u> : $R_\theta = \sum_i |\theta_i|^2$

- <u>LASSO</u> : $R_\theta = \sum_i |\theta_i|$

- <u>reduce overfitting</u>

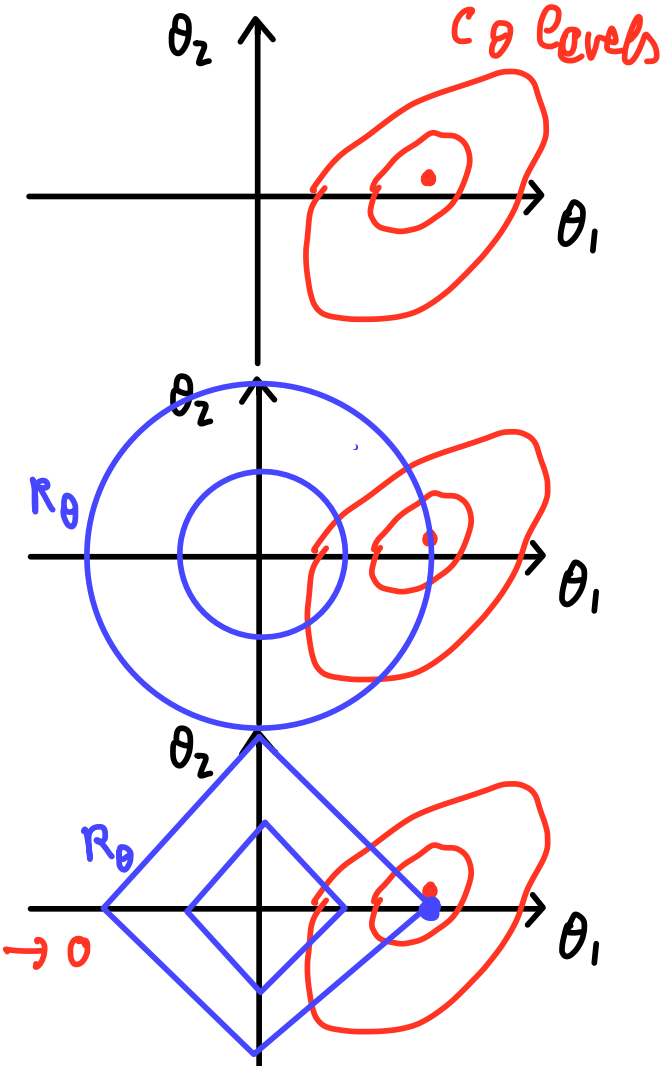
- <u>improve readability</u>

Regularization:

$$\underbrace{C_\theta + \lambda R_\theta}_{\text{to be minimized}}$$

- <u>Ridge</u> : $R_\theta = \sum_i |\theta_i|^2$

- <u>LASSO</u> : $R_\theta = \sum_i |\theta_i|$



L_1-norm in LASSO forces many weights $\to 0$